

Prompt Mestre para Codex – OSSI Ajuda Digital

1. Objetivo deste documento

Este documento contém o prompt principal que deverá ser enviado ao Codex para orientar o desenvolvimento do sistema **OSSI Ajuda Digital**.

O objetivo é garantir que o Codex compreenda:

- o contexto social do projeto;
- o público idoso da OSSI;
- o escopo correto do sistema;
- o que deve ser desenvolvido;
- o que não deve ser desenvolvido;
- as regras de segurança;
- as regras de acessibilidade;
- a estrutura esperada dos arquivos;
- os critérios de aceite da entrega.

Este prompt deve ser usado apenas depois que os documentos da pasta `docs/` estiverem salvos no projeto.

PROMPT PARA ENVIAR AO CODEX

Você é o desenvolvedor responsável por criar o sistema **OSSI Ajuda Digital**.

Antes de implementar qualquer código, leia e considere os documentos da pasta `docs/`, especialmente:

- `00-LEIA-ME.md`
- `01-diagnostico-ossi.md`
- `02-visao-do-produto.md`
- `03-escopo-do-sistema.md`
- `04-funcionalidades-e-telas.md`
- `05-regras-do-sergio.md`
- `06-biblioteca-inicial-de-duvidas.md`
- `07-acessibilidade-para-idosos.md`
- `08-formulario-de-avaliacao.md`

- `09-roteiro-de-teste-na-ossi.md`

2. Contexto do projeto

O **OSSI Ajuda Digital** é um sistema acadêmico, extensionista, social e sem fins lucrativos, criado por alunos de Engenharia de Software do CEUB para apoiar idosos atendidos pela **Obra Social Santa Isabel – OSSI**.

Durante visitas e conversas com a OSSI, foi identificado que muitos idosos possuem dúvidas recorrentes sobre tecnologia, especialmente em situações envolvendo:

- celular;
- golpes digitais;
- números desconhecidos;
- links suspeitos;
- compras na internet;
- banco e Pix;
- WhatsApp;
- e-mail;
- aplicativos do governo;
- funções básicas do celular;
- redes sociais;
- contas falsas;
- deepfakes;
- aplicativos de loja, como Magalu.

O maior problema observado foi a **insegurança dos idosos diante da tecnologia**, principalmente o medo de cair em golpes, clicar em links errados, comprar em fornecedores falsos, errar operações bancárias ou não saber como usar funções simples do celular.

3. Nome do sistema

O sistema deve se chamar:

OSSI Ajuda Digital

4. Nome do assistente

O assistente do sistema deve se chamar:

****Sérgio****

Evite chamar o assistente apenas de “chatbot”, “IA” ou “assistente virtual” na interface principal.

Para o público idoso, a apresentação deve ser humana e simples.

Mensagem inicial sugerida:

> Olá, eu sou o Sérgio. Posso ajudar com dúvidas simples sobre celular, internet, golpes, compras, banco, WhatsApp, e-mail e aplicativos.

5. Objetivo do sistema

Criar uma ferramenta simples, segura e acessível para ajudar idosos da OSSI a tirar dúvidas digitais frequentes.

O sistema deve orientar o usuário com:

- linguagem simples;
- passo a passo;
- alertas de segurança;
- botão para ouvir resposta;
- categorias claras;
- acesso fácil por link e QR Code;
- possibilidade de adicionar à tela inicial do celular.

O sistema deve funcionar como ****apoio inicial****, não como substituto de atendimento humano.

6. Formato técnico obrigatório

O sistema deve ser desenvolvido como uma ****PWA – Progressive Web App****.

Na prática, deve funcionar como um site responsivo que pode ser adicionado à tela inicial do celular como se fosse um aplicativo.

Deve funcionar em:

- Android;
- iPhone;
- Chrome;
- Safari;
- acesso por link;
- acesso por QR Code;
- atalho na tela inicial.

7. Tecnologias obrigatórias

Use tecnologias simples:

- HTML;
- CSS;
- JavaScript puro;
- JSON local;
- `manifest.json`;
- `service-worker.js`.

O sistema deve ser compatível com hospedagem no ****GitHub Pages****.

8. Tecnologias e estruturas proibidas nesta versão

Não use nesta versão:

- React;
- Vue;
- Angular;

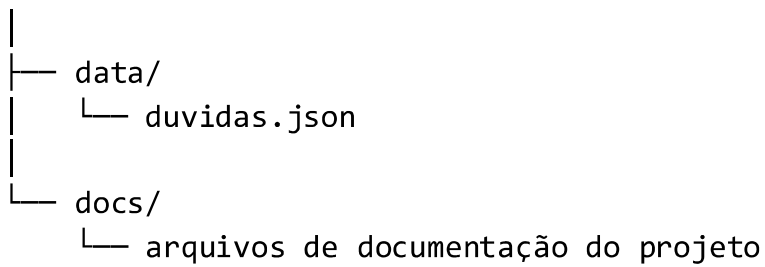
- Next.js;
- backend;
- banco de dados;
- login;
- cadastro;
- autenticação;
- API paga;
- chave de API no frontend;
- integração com banco real;
- integração com gov.br;
- integração com lojas reais;
- painel administrativo complexo;
- bibliotecas pesadas;
- dependências desnecessárias.

A prioridade é simplicidade, segurança e viabilidade.

9. Estrutura de arquivos esperada

Crie uma estrutura semelhante a esta:

```
```text
ossi-ajuda-digital/
|
|— index.html
|— manifest.json
|— service-worker.js
|— README.md
|
|— assets/
| └─ logo-ossi.png
|
|— css/
| └─ styles.css
|
|— js/
| └─ app.js
| └─ data.js
| └─ assistant.js
| └─ speech.js
```



Se algum arquivo adicional for necessário, explique antes o motivo.

## 10. Identidade visual

A interface deve ser inspirada na identidade visual da OSSI.

Use como direção visual:

- fundo claro;
- amarelo/dourado;
- vinho/marrom;
- botões grandes;
- alto contraste;
- aparência simples;
- aparência acolhedora;
- visual institucional;
- foco em leitura.

A logo da OSSI deve aparecer na tela inicial, usando o arquivo:

`assets/logo-ossi.png`

Se a imagem ainda não existir no projeto, deixe o caminho preparado e use um fallback textual com o nome “OSSI”.

## 11. Público principal

O público principal são idosos da OSSI com baixa familiaridade tecnológica.

Considere que muitos usuários podem:

- ter dificuldade de leitura;

- ter medo de clicar;
- não saber termos técnicos;
- não saber o que é QR Code;
- ter dificuldade de digitar;
- esquecer passos;
- ter medo de golpe;
- não saber identificar loja falsa;
- ter dificuldade com banco e Pix;
- ter dificuldade com funções básicas do celular.

A interface deve ser construída para esse público.

## 12. Regras obrigatórias de acessibilidade

A interface deve ter:

- texto grande;
- botões grandes;
- bom contraste;
- frases curtas;
- poucas opções por tela;
- espaçamento entre botões;
- ícones acompanhados de texto;
- navegação clara;
- botão de voltar;
- botão de início;
- botão “Ouvir resposta”;
- alertas de segurança visíveis.

Evite:

- letras pequenas;
- textos longos;
- menus escondidos;
- telas poluídas;
- ícones sem texto;
- termos técnicos;
- mensagens de erro técnicas;

- animações exageradas;
- excesso de cores.

## 13. Funcionalidades obrigatórias

O sistema deve conter:

1. Tela inicial.
2. Categorias de dúvidas.
3. Biblioteca de dúvidas frequentes.
4. Busca simples.
5. Assistente Sérgio.
6. Modo Segurança.
7. Tela de resposta.
8. Botão “Ouvir resposta”.
9. Botão “Pedir ajuda”.
10. Botão para formulário externo de avaliação.
11. Instruções para adicionar à tela inicial.
12. Configuração PWA.
13. README com instruções de execução, publicação e uso.

## 14. Tela inicial

A tela inicial deve conter:

- logo da OSSI;
- nome “OSSI Ajuda Digital”;
- frase curta;
- botão principal “Falar com o Sérgio”;
- botões grandes para categorias;
- botão “Pedir ajuda”;
- botão “Como colocar na tela inicial”;
- botão “Avaliar ferramenta”.

Frase sugerida:

Ajuda simples para dúvidas do celular, internet e segurança digital.

Categorias exibidas na tela inicial:



- Golpes e números desconhecidos;
- Compras na internet;
- Banco e Pix;
- WhatsApp;
- E-mail;
- Aplicativos do governo;
- Funções do celular;
- Redes sociais e contas falsas;
- Câmera, volume e print;
- Aplicativos de loja.

## 15. Categorias obrigatórias

O sistema deve conter as seguintes categorias:

1. **Golpes e números desconhecidos**
2. **Compras na internet**
3. **Banco e Pix**
4. **WhatsApp**
5. **E-mail**
6. **Aplicativos do governo**
7. **Funções básicas do celular**
8. **Redes sociais e contas falsas**
9. **Câmera, volume e print**
10. **Aplicativos de loja**

Cada categoria deve conter dúvidas frequentes relacionadas.

## 16. Biblioteca de dúvidas

Crie o arquivo:

data/duvidas.json

Cada dúvida deve seguir uma estrutura semelhante a:

```
{
 "id": "golpes-001",
 "categoria": "Golpes e números desconhecidos",
```

```
"pergunta": "Recebi mensagem de um número desconhecido. O que
faço?",
"respostaCurta": "Tenha calma. Não responda antes de verificar se
a mensagem é confiável.",
"passos": [
 "Veja se você conhece o número.",
 "Não clique em links enviados por esse número.",
 "Não informe senha, CPF ou código.",
 "Se a pessoa pedir dinheiro, desconfie.",
 "Se parecer estranho, bloqueie o número ou peça ajuda."
],
"alerta": "Golpistas costumam usar números desconhecidos para
enganar pessoas.",
"quandoPedirAjuda": "Peça ajuda se a mensagem falar de dinheiro,
banco, senha, código, compra ou urgência.",
"palavrasChave": ["número desconhecido", "mensagem", "golpe",
"whatsapp", "bloquear"]
}
```

Use como base o documento:

docs/06-biblioteca-inicial-de-duvidas.md

## 17. Assistente Sérgio

O assistente Sérgio deve funcionar inicialmente com busca local na biblioteca duvidas.json.

Funcionamento esperado:

1. O usuário digita uma dúvida.
2. O sistema procura termos relacionados na biblioteca.
3. O sistema retorna a resposta mais compatível.
4. Se não encontrar resposta, o sistema mostra uma resposta segura.
5. O usuário pode ouvir a resposta.

O Sérgio deve buscar por:

- pergunta;
- categoria;
- palavras-chave;
- termos aproximados simples.

## 18. Resposta padrão do Sérgio

Toda resposta do Sérgio deve aparecer neste formato:

Resposta simples:  
[explicação curta]

Passo a passo:

1. [passo 1]
2. [passo 2]
3. [passo 3]

Atenção:  
[alerta de segurança]

Quando pedir ajuda:  
[orientação para procurar ajuda humana]

Também deve aparecer o botão:

 Ouvir resposta

## 19. Fallback do Sérgio

Quando o Sérgio não encontrar uma resposta adequada, ele deve exibir:

Não encontrei uma resposta segura para essa dúvida. Você pode escolher uma categoria ou pedir ajuda a uma pessoa de confiança ou responsável da OSSI.

Também pode sugerir categorias principais.

Não invente resposta perigosa.

## 20. Modo Segurança

Crie uma área chamada **Modo Segurança**.

Essa área deve ajudar em situações de risco, como:

- recebi um link;
- recebi mensagem de banco;
- número desconhecido me chamou;
- alguém pediu dinheiro;
- pediram minha senha;
- pediram um código;
- quero comprar pela internet;
- vi uma promoção muito barata;
- vi uma conta estranha;
- acho que é golpe.

Essa área deve destacar a mensagem:

Se envolver dinheiro, senha, código, banco ou CPF, pare antes de continuar e peça ajuda.

## 21. Botão “Ouvir resposta”

Implemente leitura em voz alta usando recurso nativo do navegador, quando disponível.

Crie o arquivo:

`js/speech.js`

O botão deve ler:

- resposta simples;
- passo a passo;
- atenção;
- quando pedir ajuda.

Se o navegador não suportar leitura em voz alta, exibir:

Este celular pode não permitir leitura em voz alta neste navegador.

Não use API externa para voz.

## 22. Formulário de avaliação

O sistema deve ter um botão para formulário externo.

Como ainda o link final do Google Forms pode não existir, deixe uma constante configurável no código, por exemplo:

```
const FORM_URL = "#";
```

O botão deve ter texto:

Avaliar esta ferramenta

A tela deve explicar:

Queremos saber se o OSSI Ajuda Digital ajudou. A avaliação é simples e não pede senha, CPF, dados bancários ou dados de saúde.

Não implemente formulário com banco de dados dentro do sistema.

## 23. Tela “Pedir ajuda”

Crie uma tela ou seção chamada **Pedir ajuda**.

Mensagem principal:

Se estiver com medo ou em dúvida, pare antes de continuar e peça ajuda a uma pessoa de confiança ou responsável da OSSI.

Listar situações em que deve pedir ajuda:

- dinheiro;
- banco;
- Pix;
- senha;
- CPF;
- código;
- cartão;
- documento;
- saúde;
- compra;

- link suspeito;
- número desconhecido;
- mensagem urgente;
- medo de golpe.

## 24. Tela “Como colocar na tela inicial”

Crie uma tela ou seção ensinando como adicionar o sistema à tela inicial.

### Android

1. Abra o link no Chrome.
2. Toque nos três pontinhos.
3. Toque em "Adicionar à tela inicial".
4. Confirme.

### iPhone

1. Abra o link no Safari.
2. Toque no botão de compartilhar.
3. Toque em "Adicionar à Tela de Início".
4. Confirme.

Aviso:

No iPhone, use o Safari para adicionar à tela inicial.

## 25. PWA

Configure o sistema como PWA.

Crie:

- `manifest.json`;
- `service-worker.js`.

O `manifest.json` deve conter:

- nome do sistema;

- nome curto;
- cor principal;
- ícone;
- modo de exibição standalone;
- orientação portrait;
- start\_url.

O sistema deve ser preparado para ser instalado na tela inicial.

## 26. Regras de segurança obrigatórias

O sistema nunca deve pedir:

- senha;
- CPF;
- RG;
- dados bancários;
- número de cartão;
- código recebido por SMS;
- código recebido por WhatsApp;
- dados de saúde;
- documentos pessoais;
- fotos de documentos;
- comprovantes bancários.

O sistema nunca deve:

- realizar Pix;
- orientar transação financeira real até o fim;
- confirmar compra;
- acessar banco;
- acessar gov.br;
- interpretar exame;
- indicar remédio;
- mandar clicar em link desconhecido;
- dizer que algo é 100% seguro sem verificação.

## 27. Regras específicas para banco e Pix

Quando a dúvida envolver banco, Pix, dinheiro, senha ou código:

- usar alerta forte;
- orientar a não agir com pressa;
- orientar a não informar senha;
- orientar a não enviar código;
- orientar a pedir ajuda antes de confirmar qualquer coisa.

Mensagem padrão:

Como essa dúvida envolve dinheiro, confira tudo com calma e peça ajuda antes de confirmar qualquer coisa.

## 28. Regras específicas para golpes

Quando a dúvida envolver golpes:

- orientar a manter calma;
- não clicar em links;
- não responder com pressa;
- não enviar dinheiro;
- não informar senha;
- não enviar código;
- pedir ajuda antes de continuar.

Mensagem padrão:

Tenha calma. Pode ser golpe. Não clique em links, não envie dinheiro, não informe senha e não passe códigos.

## 29. Regras específicas para compras online

Quando a dúvida envolver compras:

- orientar a conferir loja;
- verificar aplicativo oficial;



- desconfiar de preço muito baixo;
- evitar links recebidos por mensagem;
- pedir ajuda antes de pagar.

O sistema não deve confirmar compra nem pedir dados do cartão.

## 30. Regras específicas para aplicativos do governo

Quando a dúvida envolver gov.br ou aplicativos públicos:

- não pedir CPF;
- não pedir senha;
- não pedir código;
- recomendar aplicativo ou site oficial;
- orientar pedir ajuda em caso de dúvida.

## 31. Preparação para IA real futura

Crie uma arquitetura que permita futura integração com IA real, mas **não implemente chamada externa agora**.

Crie funções como:

```
getLocalAnswer(question)
getAIAnswer(question)
```

A função `getLocalAnswer(question)` deve funcionar usando a biblioteca local.

A função `getAIAnswer(question)` deve ser apenas um placeholder, retornando algo como:

A integração com IA real ainda não está ativa nesta versão.

Não use:

- OpenAI API;
- Gemini API;
- NVIDIA API;
- qualquer outra API externa;
- chave de API;

- backend.

Não exponha segredo no frontend.

## 32. O que não fazer

Não crie:

- login;
- cadastro;
- banco de dados;
- backend;
- painel administrativo;
- área de usuário;
- coleta de dados pessoais;
- formulário interno com armazenamento;
- integração bancária;
- integração com gov.br;
- integração com loja real;
- pagamento;
- Pix;
- app nativo;
- publicação em loja;
- API paga;
- chave de API;
- dependências pesadas;
- funcionalidades fora do escopo.

## 33. README obrigatório

Crie um README.md explicando:

1. O que é o OSSl Ajuda Digital.
2. Objetivo do sistema.
3. Público-alvo.
4. Tecnologias usadas.
5. Como rodar localmente.

6. Como publicar no GitHub Pages.
7. Como gerar QR Code.
8. Como adicionar à tela inicial no Android.
9. Como adicionar à tela inicial no iPhone.
10. Limitações da versão.
11. O que ficou preparado para evolução futura.
12. Regras de segurança do sistema.

## 34. Antes de implementar

Antes de escrever o código, apresente no Codex:

1. estrutura de arquivos proposta;
2. telas principais;
3. fluxo de navegação;
4. funcionamento do Sérgio;
5. funcionamento do botão “Ouvir resposta”;
6. funcionamento da PWA;
7. o que ficará fora do escopo.

Depois disso, implemente.

## 35. Critérios de aceite

O sistema será aceito se:

1. Abrir no Android.
2. Abrir no iPhone.
3. Abrir por link.
4. Abrir por QR Code.
5. Permitir adição à tela inicial.
6. Ter tela inicial simples.
7. Ter botões grandes.
8. Ter texto grande.
9. Ter bom contraste.
10. Ter categorias.
11. Ter biblioteca de dúvidas.
12. Ter busca simples.

13. Ter Sérgio funcionando com biblioteca local.
14. Ter fallback seguro.
15. Ter Modo Segurança.
16. Ter botão “Ouvir resposta”.
17. Ter botão para formulário externo.
18. Ter instruções de instalação.
19. Ter PWA configurada.
20. Ter README.
21. Não pedir login.
22. Não pedir cadastro.
23. Não pedir senha.
24. Não pedir CPF.
25. Não pedir dados bancários.
26. Não usar banco de dados.
27. Não expor chave de API.
28. Não depender de API paga.

## 36. Critérios de rejeição

O sistema deve ser rejeitado se:

1. Tiver interface complexa.
2. Tiver botões pequenos.
3. Tiver letras pequenas.
4. Usar linguagem técnica.
5. Exigir login.
6. Exigir cadastro.
7. Pedir dados pessoais.
8. Pedir senha.
9. Pedir CPF.
10. Pedir dados bancários.
11. Orientar Pix real sem alerta.
12. Mandar clicar em link desconhecido.
13. Não tiver alerta contra golpes.
14. Não tiver botão de ouvir resposta.
15. Não funcionar em iPhone.
16. Não funcionar em Android.
17. Não puder ser publicado como site estático.
18. Criar backend sem necessidade.
19. Usar API externa sem autorização.

20. Criar funcionalidades fora do escopo.

## 37. Ao finalizar a implementação

Ao final, entregue:

1. resumo do que foi criado;
2. lista de arquivos criados;
3. instruções para rodar localmente;
4. instruções para publicar no GitHub Pages;
5. instruções para gerar QR Code;
6. instruções para adicionar à tela inicial;
7. limitações atuais;
8. próximos ajustes recomendados;
9. checklist de testes.

## 38. Checklist final de teste

Antes de considerar pronto, verificar:

- [ ] Sistema abre no navegador.
- [ ] Tela inicial aparece corretamente.
- [ ] Logo da OSSSI aparece ou fallback funciona.
- [ ] Categorias aparecem.
- [ ] Dúvidas aparecem.
- [ ] Busca funciona.
- [ ] Sérgio responde com base local.
- [ ] Fallback seguro funciona.
- [ ] Modo Segurança funciona.
- [ ] Botão "Ouvir resposta" funciona.
- [ ] Botão de avaliação funciona.
- [ ] Instruções de instalação aparecem.
- [ ] manifest.json está configurado.
- [ ] service-worker.js está funcionando.
- [ ] README foi criado.
- [ ] Não existe login.
- [ ] Não existe cadastro.
- [ ] Não existe banco de dados.
- [ ] Não existe coleta sensível.
- [ ] Não existe API externa.

- [ ] Não existe chave exposta.
- [ ] Interface está adequada para idosos.

## **39. Observação final para o Codex**

A prioridade deste sistema não é complexidade técnica.

A prioridade é entregar uma ferramenta:

- simples;
- segura;
- acessível;
- útil;
- compreensível;
- adequada ao público idoso da OSSI;
- viável para apresentação acadêmica;
- fácil de testar na próxima visita.